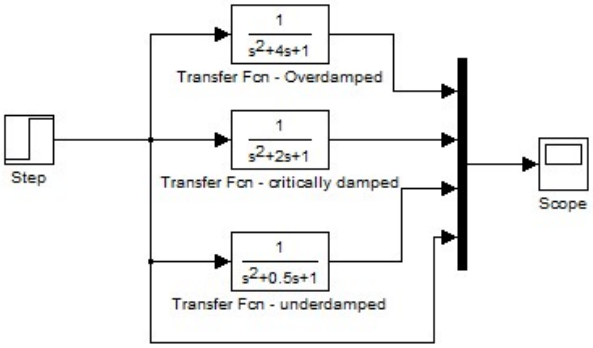
UNIVERSITÉ IBN ZOHR
FACULTÉ DES SCIENCES
AGADIR

جامعة ابن زهر
كلية العلوم
أكادير

Département de la Physique

Travaux Pratiques d'Automatique

ES6 & EER6



Responsable :
Prof. Mme F. EL GUEZAR

Année universitaire 2025-2026

Préface

Ce fascicule de travaux pratiques constitue une modeste contribution de notre part dans le cadre de l'élaboration d'un support pédagogique destiné aux étudiants de la section Matière Physique, pour les filières ES6 et EER6. Ce polycopié de travaux pratiques présente les éléments de base nécessaires à la compréhension des systèmes asservis linéaires. Il s'adresse, en général, aux étudiants de troisième année préparant une licence en automatique.

Auteur: Prof. Mme Fatima EL GUEZAR

19 Février 2026

I HEAR, I FORGET
I SEE, I REMEMBER
I DO, I UNDERSTAND

PRACTCAL APPROACH IS PROBABLY THE BEST APPROACH TO GAIN A CLEAR INSIGHT

Fonctionnement Général

Le cycle de TP se déroule sur 4 séances. Il comprend 4 manipulations de 4 heures chacune. Les différentes manipulations sont décrites par la liste ci-dessous :

TP N° 1: Prise en main en Matlab et Simulink 4

TP N° 2 : Analyse des systèmes linéaires de 1er et 2ème ordre 15

TP N° 3: Correcteurs P et PI 18

Consignes aux étudiants pour le bon déroulement des travaux pratiques

1. Préparation préalable obligatoire :
Avant chaque séance de travaux pratiques (TP), les étudiants doivent préparer le sujet du TP et répondre aux questions théoriques liées. Une révision du cours est indispensable pour bien comprendre les concepts abordés.
2. Respect des groupes de TP :
Chaque étudiant doit obligatoirement respecter le groupe de TP auquel il appartient. Aucune modification de groupe ne sera tolérée sans autorisation préalable.
3. Matériel personnel :
Chaque étudiant est tenu de se munir de son propre polycopié. Le partage de documents ou de supports pendant la séance ne sera pas permis.

Le respect de ces consignes est essentiel pour assurer le bon déroulement des séances et optimiser l'apprentissage de chacun.

Notation

Chaque manipulation donne lieu à un compte-rendu par groupe à rendre, et qui fera l'objet d'une note qui prend en compte la préparation (contrôlée en début de séance), la participation des étudiants au déroulement du TP, les résultats obtenus et leur interprétation. La moyenne des 3 notes obtenues (une par TP) + une colle de TP, constituera la note finale associée à ce cycle d'enseignement.

TP N° 1: Prise en main en Matlab et Simulink

Objectif

Ce TP a pour but de se familiariser avec l'usage du logiciel Matlab ainsi que sa Toolbox graphique Simulink dans le cadre d'une utilisation pour les asservissements.

Dans un premier temps, on abordera quelques exemples pour mieux utiliser Matlab: la saisie des données sous Matlab, le traçage des courbes, description d'un système sous forme de fonction de transfert. Nous étudierons ensuite, les réponses temporelles et fréquentielles des systèmes de 1er et 2ème ordre.

A. Travail sous Matlab

I. PRESENTATION DU LOGICIEL :

MATLAB (**MAT**rix **LAB**oratory) est un système interactif et convivial de calcul numérique et de visualisation graphique destiné aux ingénieurs et scientifiques. Il possède un langage de programmation à la fois puissant et simple d'utilisation.

Dans MATLAB, l'élément de base est la matrice. L'utilisateur ne s'occupe pas des allocations mémoire ou de redimensionnement comme dans les langages classiques. Les problèmes numériques peuvent être résolus en un temps record, qui ne représente qu'une fraction infime du temps à passer avec d'autres langages comme le Basic, C, C++ ou le Fortran.

En complément de MATLAB, l'outil additionnel SIMULINK est proposé pour la modélisation et la simulation de systèmes dynamiques en utilisant une représentation de type graphique (schéma bloc).

N'oubliez pas l'aide en ligne dans Matlab avec la commande **: help *instruction***

II. Opérations sur MATLAB

1) Variables

Une variable dans MATLAB peut être un scalaire, un nombre complexe, un vecteur ou une matrice. Un nom de variable peut comporter jusqu'à 31 caractères et doit commencer par une lettre. Il peut contenir des lettres, des chiffres et des caractères de soulignement.

Si une entrée de données, une instruction ou toute commande n'est pas terminée par un point-virgule, le résultat de l'instruction est toujours affiché.

a) Scalaires

Un nombre entier peut être attribué à un nom de variable comme suit:

```
>> w = 5
>> p = -3
```

Un nombre réel est entré en spécifiant un point décimal, ou l'exposant:

```
>> q = 2.35
>> a = 5.2e-3
```

Lors de la saisie d'un nombre complexe, la partie réelle est saisie en premier, suivie de la partie imaginaire:

```
>> p = 2 + 4*i
>> q = 12*exp (i*2)
```

b) Matrices

```
>>A=[1 2 3; 4 5 6;7 8 9; 10 11 12] ↵ %matrice rectangle 4*3
>> size(A); %dimension de la matrice
```

>>A(i,j); % élément ligne i colonne j
>>A(:,n); % n^{ième} colonne
>>A(p,:); % p^{ième} ligne
>>A(i:j,:); % sous matrice des lignes i à j
>>A(i:j,k:l); % sous matrice des lignes i à j colonnes k à l

c) vecteurs

Un vecteur ligne est entré en séparant éventuellement les éléments par des virgules. Par exemple, le vecteur p = [2 4 6] est entré comme
>> p = [2, 4, 6] ↵ **ou** >> p = [2 4 6] ↵
De même, un vecteur colonne est entré en séparant les éléments par des points-virgules.
>> q = [3; 6; 9] ↵
Un vecteur peut être transposé en utilisant le caractère '. Par exemple, le vecteur q ci-dessus peut être transposé comme
>> a = [3; 6; 9]' ↵
>> p(i) ; %i^{ème} élément ATTENTION le premier indice est à i=1
>> length(p) %dimension du vecteur

Saisie d'un vecteur de temps

Ces vecteurs peuvent être créés en spécifiant la valeur initiale, la valeur finale et l'incrément. La syntaxe est la suivante: vecteur = valeur_initial:incrément:valeur_finale. Par défaut, le pas est égal à 1.
Par exemple,
>> T = 0:1:10 ↵
Si l'incrément est 1, il peut être omis. Ainsi, le vecteur ci-dessus peut également être créé avec la déclaration
>> T = 0:10 ↵

On peut utiliser aussi la fonction prédéfinie linspace(début,fin,nbre_pts) :
>> v = linspace(0,10,1000) ↵
Cette fonction crée un vecteur de 1000 points avec des valeurs allant de 0 à 10 également espacées.

Saisie d'un vecteur de pulsation

Soit w, un vecteur définissant un intervalle de fréquences de 0.1rd/s à 100rd/s. Cet intervalle de valeurs nous permettra de tracer des lieux de Bode. On travaillera en échelle logarithmique. On utilise la fonction logspace(début,fin,nbre_pts)
>> w=logspace(-1,2,300) ↵
Cette fonction crée un vecteur de 300 points sur un intervalle de 10^{-1} à 10^{2} avec un espacement logarithmique.

Polynômes :

Un polynôme est défini en utilisant un vecteur contenant les coefficients du polynôme. Par exemple, le polynôme

$$P1 = 6x^4 - 2x^3 + 5x^2 - 2x + 1$$

est défini comme
>> P1 = [6 -2 5 -2 1] ↵
Il est important que toutes les puissances du polynôme soient spécifiées. Les coefficients des puissances manquantes doivent être spécifiés à zéro.

La commande ***roots*** calcule les racines d'un polynôme. Les racines de P1=0 se trouvent comme suit:

>> r = roots(P1)↵

Le polynôme a quatre racines complexes.

Considérons maintenant un autre polynôme

$$P2 = 2x^4 - x^3 + 2x^2 - x + 3.$$

Le produit des polynômes P1 et P2 se présente comme suit:

>> P2 = [2 -1 2 -1 3]; ↵

>> P3 = conv(P1,P2) ↵

La commande ***poly*** permet la création d'un polynôme à partir de ses racines.

Exemple, le polynôme dont les racines sont 2 et 5 se retrouve comme suit:

>> poly([2 5]) ↵

Ainsi, l'équation du polynôme requis est

$$F(x) = x^2 - 7x + 10.$$

2) Représentations graphiques

Les bibliothèques de Matlab proposent un très grand nombre de fonctions pour la manipulation d'objets graphiques. Nous ne présentons ici que quelques principes de base, utiles pour la visualisation de courbes. Si nous nous concentrons particulièrement sur la représentation graphique à 2 dimensions, il est possible d'aller bien plus loin : graphismes 3D…

Graphiques 2D

Le tracé d'une courbe s'effectue à partir de la commande plot(). Celle-ci prend en paramètres deux vecteurs et affiche sur un graphique à deux axes chaque couple de points (de même indice). Par exemple, plot(x,y) marquera un point pour chaque couple [x(i),y(i)] avec i allant de 0 à length(x). On représente ainsi les valeurs de y en fonction des valeurs de x. La fonction renvoie une erreur si x et y ne sont pas de même longueur.

Traçons la fonction sinus dans l'intervalle $[-\pi, \pi]$ avec un pas de 0.01.

>> x = -pi : .01:pi; ↵

>> y = sin(x); ↵

>> plot(x,y) ↵

Le pas étant faible, la courbe semble parfaitement tracée. Bien évidemment, si l'on diminue le nombre de points (le pas est augmenté), la courbe apparaitra plus saccadée.

>> x = -pi : .6 : pi; ↵

>> y = sin(x); ↵

>> plot(x,y) ↵

La commande plot prend un troisième argument permettant de spécifier la couleur du tracé et le symbole de représentation. Retraçons l'exemple précédent en rouge avec des étoiles pour chaque point.

>> x = -pi : .6 : pi; ↵

>> y = sin(x); ↵

>> plot(x,y,'r *') ↵

Différentes options sont disponibles (consulter le help plot).

À chaque nouvelle commande plot, la figure est remplacée. Pour garder plusieurs courbes, il faut autoriser la superposition de graphique à l'aide de la commande hold on. Les plots suivants se superposeront jusqu'à la désactivation hold off ou la fermeture de la fenêtre.

>> x = linspace(-20,20,1000); ↵

```
>> y = x.*sin(x); ↵
>> plot(x,y) ↵
>> hold on↵
>> y2 = -x; ↵
>> plot(x,y2,'r') ↵
>> hold off↵
```

Il est également possible de tracer plusieurs courbes sur plusieurs fenêtres. Pour cela, une nouvelle fenêtre (objet graphique figure) doit invoquée avant l'appel à la fonction plot correspondante.

```
>> plot(x,y) ↵
>> figure(2) ↵
>> plot(x,y2,'r') ↵
```

Affichage de plusieurs courbes, pour cela la commande subplot divise la fenêtre graphique :

```
> > subplot(2,1,1) ↵
> > plot(x,y) ↵
> > subplot(2,1,2) ↵
 > > plot(x,y2) ↵
```

D'autres fonctions permettent une représentation différente des données, par exemples sous forme discrète (stem), d'histogramme (bar), de camembert (pie), d'escalier (stairs), avec échelle logarithmique (semilogx, semilogy)…

La mise en forme d'une représentation graphique, c'est-à-dire l'insertion de labels, légende, le dimensionnement des axes, peut être éditée de deux manières. La méthode la plus simple utilise directement les menus de l'interface de la figure (Edit et Insert). Toutes ces manipulations sont également réalisables à partir des commandes lors du développement de programmes. Voici quelques exemples parmi les manipulations les plus simples :

```
>> xlabel('valeur x')
>> ylabel('valeur y')
>> title('mon graphique')
>> legend('ma courbe')
>> grid on
>> axis([xmin xmax ymin ymax])
```

Travail demandé 1:

Tracez la fonction f(t)=sin(t) pour t variant de 0 à 50s pour un pas de 0.01s et afficher la courbe dans une fenêtre graphique. Sur la même fenêtre, tracez la fonction sin (t/2).

III. BOÎTE À OUTILS CONTROL SYSTEM TOOLBOX

Control System Toolbox est une collection d'algorithmes qui utilise les fonctions MATLAB pour fournir des fonctions spécifiques à l'ingénierie de contrôle. Dans cette section, nous examinerons brièvement certaines des fonctions importantes de la Control System Toolbox pour les systèmes à temps continu

Systèmes à temps continu

Considérons un système à temps continu à une entrée unique et sortie unique (SISO) avec la fonction transfert en boucle ouverte :

$$\frac{Y(s)}{U(s)} = \frac{3}{s^2 + 3s + 9}$$

1) Saisie d'une fonction de transfert

La fonction de transfert du système peut être définie en termes de numérateur et de dénominateur polynomial:

>> num = [0 0 3];

>> den = [1 3 9];

Il existe généralement deux façons de représenter la fonction de transfert dans Matlab:

a) Utilisation de la commande tf:

La fonction de transfert est donnée par:

>> G = tf(num,den) ↵

Travail demandé 2: Fonction de transfert avec la commande tf (num, den)

Nous avons trois avec les FT suivantes, nous allons entrez-les en utilisant la commande tf(num, den)

$$H_1(s) = \frac{s+1}{s+2} \quad , \quad H_2(s) = \frac{s^2 + 2s + 3}{s^3 + 4s^2 + 5s + 6} \quad , \quad H_3(s) = \frac{3s^2}{2s^2 + 6s + 3}$$

b) Utilisation de modèles ZPK (Zero-Pole-Gain)

Une autre façon de représenter la FT est de prendre en compte numérateur et dénominateur:

$$G(p) = K\,\frac{(p-z_1)\cdots(p-z_n)}{(p-p_1)\cdots(p-p_n)}$$

L'avantage de cette forme est que les zéros (z) et les pôles (p) de la FT sont faciles à visualiser.

La syntaxe pour spécifier ce type de modèles est:

>>G = zpk (z, p, k)

où z est les zéros de la FT sous forme vectorielle, p les pôles sous forme vectorielle et k le gain.

Travail demandé 3: Entrez les FTs suivantes définies comme modèles ZPK:

$$G(s) = \frac{-2s}{(s-2)(s^2-2s+2)} \quad , \quad W(s) = \frac{(s+1)(s+3)}{s(s+2)(s+4)}$$

Les propriétés d'un modèle de FT sont obtenues en tapant :

>> get(G) ↵

Modification des champs à l'aide de la fonction set

>>set(G,'Variable','p') ↵ **ou** >>G.Variable='p' ↵

2) Connexions de systèmes

Des systèmes complexes peuvent être construits à partir de sous-systèmes élémentaires. Voici les trois opérations de base pour l'interconnexion de modèles.

a) Fonctions de transfert connectées en série et en parallèle.

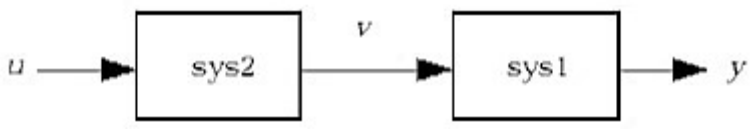
Considérons deux fonctions de transfert connectées en série sys1(s)sys2(s). La fonction de transfert globale peut être obtenue en series(G, H) où G et H sont les fonctions de transfert sys1(s) et sys2(s), respectivement, dans la représentation MATLAB.

Par exemple, si :

$$sys1(s) = \frac{1}{s^2+3s+4} \qquad \text{et} \qquad sys2(s) = \frac{2}{s+5}$$

Alors G(s) H(s) peut être obtenu par :

>> sys1 = tf(1,[1 3 4]); ↵



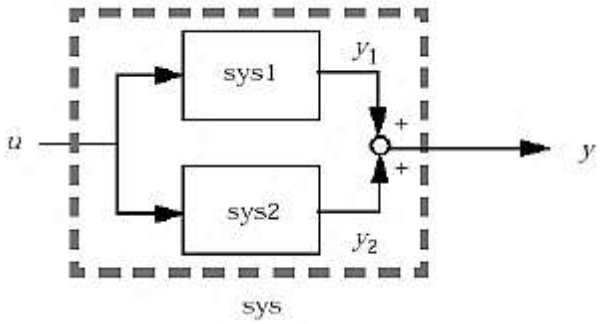
>> sys2= tf(2,[1 5]); ↵
>> sys = series(sys1,sys2) ↵
ou bien

>> sys=sys1*sys2 ↵

De même, l'instruction parallel peut être utilisée pour réduire les fonctions de transfert connectées en parallèle.
>> sys = parallel(sys1,sys2) ↵

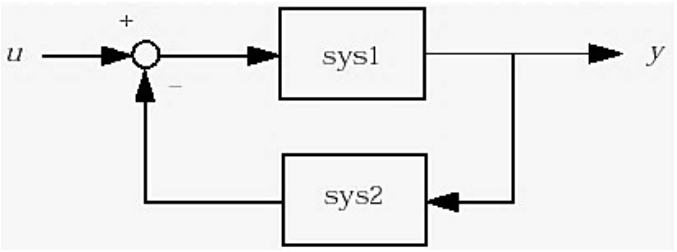
ou bien

>> sys=sys1+sys2 ↵



b) Bouclage d'un système

La fonction de transfert du système en boucle fermée peut être obtenue à l'aide de la commande feedback(). La syntaxe de base est la suivante:
>> sys = feedback((sys1, sys2) ↵
Le système résultant sys a l'entrée u et la sortie y. Les systèmes sys1 et sys2 peuvent être de types différents (par exemple sys1 TF et sys2 ZPK), mais les deux doivent être continus ou discrets.



Par défaut, Matlab suppose une rétroaction négative, pour appliquer une rétroaction positive la syntaxe est:
>> w = feedback(sys1, sys2, + 1) ↵
Pour plus de détails sur cette commande, écrivez dans Matlab: **help feedback**

c) Fonctions de transfert factorisées.

Si une fonction de transfert est sous forme factorisée, elle peut être saisie en utilisant la commande conv. Par exemple, si :

$$G(s) = \frac{(s\ +\ 4)}{(s\ +1)(s\ +\ 2)}$$

Alors elle peut être entrée dans MATLAB en tant que
>> num = [1 4]; ↵
>> den = conv([1 1], [1 2]); ↵
De même, pour la fonction de transfert

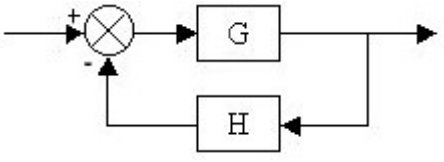
$$G(s) = \frac{2}{(s\ +\ 1)(s\ +\ 2)(s\ +\ 4)}$$

On peut écrire :
>> num = 2; ↵
>> den = conv([1 1],conv([1 2], [1 4])); ↵

Travail demandé 4: Systèmes bouclés dans Matlab avec feedback().
Calculez le système de rétroaction de la figure pour les cas suivants:

a) $G(s) = \frac{10}{s^2+5s+3}$, $H(s) = \frac{(s+4)}{(s+1)(s+2)}$

b) $G(s) = \frac{1}{(s+5)(s+2)}$, $H(s) = 1$



3) Recherche de pôles et de zéros.

Puisque les pôles et les zéros sont simplement les racines du dénominateur et du numérateur, respectivement, La commande roots est utilisée.

Par exemple, quels sont les pôles et les zéros de G(p). Taper :

>> pôles = roots (gden) ↵

>> zéros =roots (gnum) ↵

Si vous avez défini un système comme, par exemple, sys=tf (num, den), alors les commandes suivantes donnent les mêmes résultats que ci-dessus :

>> pole (sys) ↵

>> zero (sys) ↵

4) Calcul de la transformée Inverse de Laplace avec MATLAB

La commande de MATLAB «residue» est utilisée pour obtenir la transformée de Laplace inverse d'une fonction de transfert en trouvant les coefficients de la décomposition en éléments simples. L'expansion de fraction partielle est supposée être au format suivant:

$$Y(s) = \frac{r(1)}{s - p(1)} + \frac{r\,(2)}{s - \,p(2)} + \frac{r\,(3)}{s - p(3)} + \cdots + \frac{r(n)}{s - p(n)} + \,k(s)$$

Par exemple, supposons que l'on veut trouver les coefficients A, B et C de de la décomposition en éléments simples de la fonction de transfert suivante :

$$Y(s) = \frac{1}{(s+1)\,(s+2)\,(s+3)} = \frac{A}{s+1} + \frac{B}{s+2} + \frac{C}{s+3}$$

Les commandes MATLAB demandées sont :

>> num = [1]; ↵

>> den = conv ([1 1], conv ([1 2], [1 3])); ↵

>> [r, p, k] = residue (num, den) ↵

La décomposition en éléments simples de Y est alors

$$Y(s) = \frac{0{,}5}{s+1} - \frac{1}{s+\,2} + \frac{0{,}5}{s+\,3}$$

<u>Travail demandé 5:</u> commande residue(num, den)

Étant donné la fonction suivante, décomposer en fractions partielles:

$$F(s) = \frac{s+3}{s^2 + 3s + 2}$$

Analyse des systèmes

1) Réponses temporelles

Un ensemble de fonctions permettent de calculer et tracer la réponse d'un système à une entrée donnée. Plus particulièrement, la fonction step simule la réponse indicielle et impulse la réponse impulsionnelle d'un système donné en argument.

a) Réponse indicielle

L'analyse d'un système par rapport à un échelon unitaire est la plus utilisée dans l'automatique. La commande utilisée pour effectuer cette analyse avec MATLAB est **step**. La syntaxe est:

>> step (sys)

Cette fonction dessinera la réponse d'un SLI, à une entrée step unitaire. Le modèle peut être continu ou discret, SISO ou MIMO. La réponse pour un système à entrées multiples, ce sera

une collection de réponses étape pour chaque entrée. Dans ce cas, la durée de la simulation est déterminée automatiquement par MATLAB.
>> step (sys, t)
De cette façon, nous déterminons explicitement le temps de simulation. On peut spécifier t comme temps de fin ou au moyen d'un vecteur du type:
t = 0: dt: Tfinal où dt est le temps d'échantillonnage.
>> step (sys1, sys2, ..., sysN, t)
La syntaxe pour dessiner la réponse indicielle de divers systèmes SLI dans la même figure :
>> step (sys1, sys2, ..., sysN)
>> step (sys1, 'y:', sys2, 'g--') % Comme tout graphique dans MATLAB, nous pouvons spécifier des couleurs et des styles de ligne.
>> [y, t] = step(sys) % De cette façon, nous obtenons la réponse indicielle y et le vecteur temps t, utilisé en simulation. Aucun graphique n'est dessiné.

La valeur en régime permanent ou le gain statique de la réponse indicielle est obtenue comme suit:
>> ys = dcgain(num,den)

Pour lire des valeurs sur la courbe et cliquer avec la souris sur 3 points à mesurer (pas de point-virgule sinon commande inopérante ici)
>>[x,y]=ginput(3)

La commande line([x1,x2],[y1,y2]) trace une ligne du point (x1,y1) au point (y1,y2).
line([0 10],[1.05 1.05])
line([0 10],[0.95 0.95])
Ces instructions permettent de tracer deux lignes horizontales et de faire apparaître le "tuyau des réponses à 5%" pour mesurer le temps de réponse $tr_{5\%}$

b) Réponse impulsionnelle

En pratique, la fonction d'impulsion est physiquement impossible. Cependant, l'analyse des impulsions est un outil couramment utilisé, car grâce à elle, il est possible de vérifier la stabilité et la vitesse d'un système.
Pour voir la réponse d'un système à une entrée impulsion, dans MATLAB, nous utilisons la commande **impulse** de syntaxe identique à celle de la commande step.

c) Réponse à un signal quelconque

Il faut d'abord définir le temps :
t=0:0.1:10; % la variable t prend des valeurs de 0 à 10 par pas de 0.1

Une rampe :
ramp=t;
y=lsim(num,den,ramp,t);
plot(t,y)

Une sinusoïde :
Sinus=sin(t); % ne pas employer sin comme nom car c'est une fonction de Matlab et cela entraînerait une "plante" du programme
z=lsim(num,den,Sinus,t);
plot(t,z)

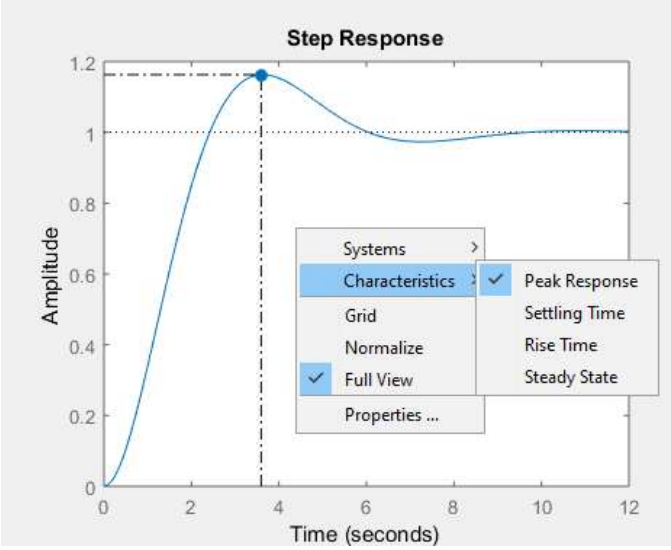
2) Systèmes de premier et deuxième ordre

Comme nous l'avons vu que les systèmes linéaires sont définis par une équation différentielle. Ces équations peuvent être classées en premier ordre, deuxième ordre, troisième ordre ..., selon la dérivée d'ordre le plus élevé dans l'équation. Dans cette section, nous étudierons le comportement temporel des systèmes de premier ordre et de second ordre. Pour effectuer l'analyse temporelle de tout système, nous effectuerons les étapes suivantes:

1) Modélisez le modèle selon les commandes vues : tf, zpk.

2) Appliquer les analyses temporaires que nous jugeons appropriées, selon l'étape d'excitation, impulsion, rampe ... Pour toute excitation appliquer les commandes lsim, gensig.

3) Visualisez les résultats obtenus. Ci-dessous, nous expliquons comment obtenir les caractéristiques du temps de réponse, du temps de montée, du dépassement... d'un système. Une fois que nous avons obtenu la réponse du système, nous double-cliquons sur la figure ou en cliquant avec le bouton droit, sélectionnez Propriétés ... De cette façon, nous accéderons à un tableau des propriétés du graphique.



Dans cette case, vous pouvez modifier:

-Dessin d'étiquettes: le titre, les étiquettes en x et en y.

-Limites: nous pouvons mettre à l'échelle automatiquement ou entrer une nouvelle échelle pour x et y.

-Style: Ici, nous pouvons activer la grille (afficher la grille), changer les propriétés de la police et des couleurs du graphique.

Mais la propriété la plus intéressante est les caractéristiques de la réponse. Cette option n'est disponible qu'en cas de step ou impulse. Les options sont:

- **Peak Reponse:** affiche t pic le temps de pic du système.
- **Settling Time:** affiche le temps de réponse. Par défaut, il donne le temps de réponse à 2%, c'est-à-dire lorsque la réponse atteint 98% de la valeur finale. Ici, nous pouvons spécifier d'autres pourcentages.
- **Rise Time:** affiche le temps de montée. Par défaut, il est défini entre 10% et 90%.
- **Steady State:** affiche l'état stable de la réponse (La valeur finale).

Pour la réponse impulsionnelle, nous n'avons que la réponse de crête et la stabilisation temps.

B. Travail sous SIMULINK

Le Toolbox SIMULINK (Simulation and Link), est une fonction lancée depuis Matlab qui donne accès à une description graphique des problèmes avec application au traitement du signal, l'automatique classique, la logique floue...

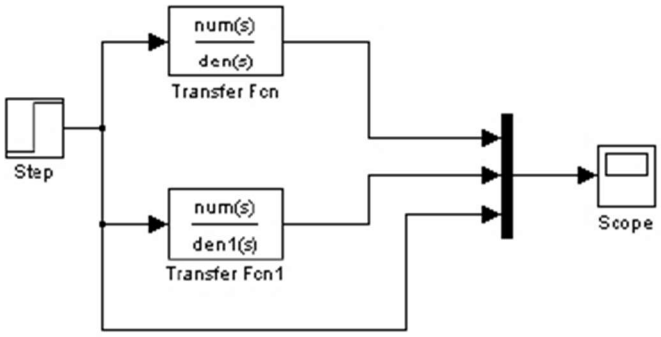
La base est une description en schéma-bloc hiérarchique, chaque bloc étant une fonction décrite en langage Matlab ou à partir d'autres blocs. Pour l'utilisateur, la description et l'exécution de Matlab est transparente dans les problèmes usuels. Il ne se préoccupe que de la construction graphique de son problème avec éventuellement création de nouveaux blocs pour les problèmes avancés. La sauvegarde des applications Simulink est un fichier texte d'extension .mdl.

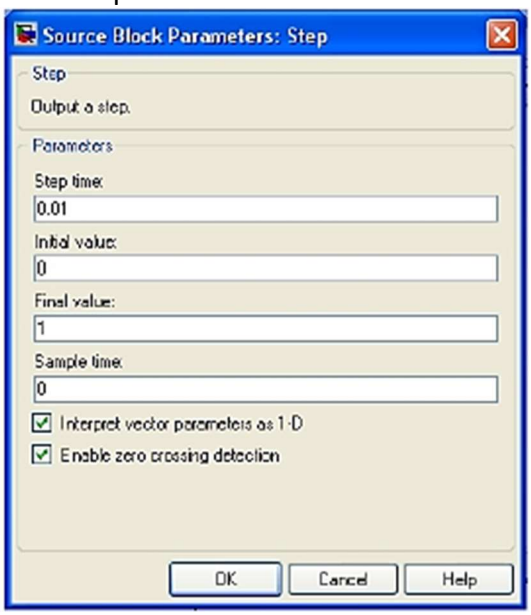
a) Réponse indicielle avec Simulink

Nous devons déterminer les fonctions de transfert dans l'espace de travail pour pouvoir utiliser dans Simulink. Nous utiliserons les mêmes fonctions de transfert que l'exemple précédent pour faire une comparaison des graphes résultants.

>> num = [1];
>> den = [1 0.5 1];
>> den1 = [1 0.5 4];

Schéma fonctionnel dans Simulink et la configuration de bloc Step





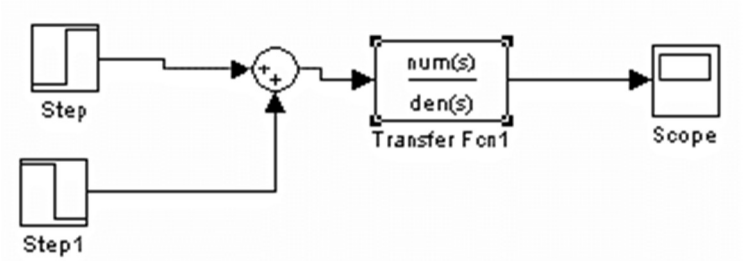
b) Réponse impulsionnelle avec SIMULINK

On définit d'abord une fonction de transfert dans l'espace de travail

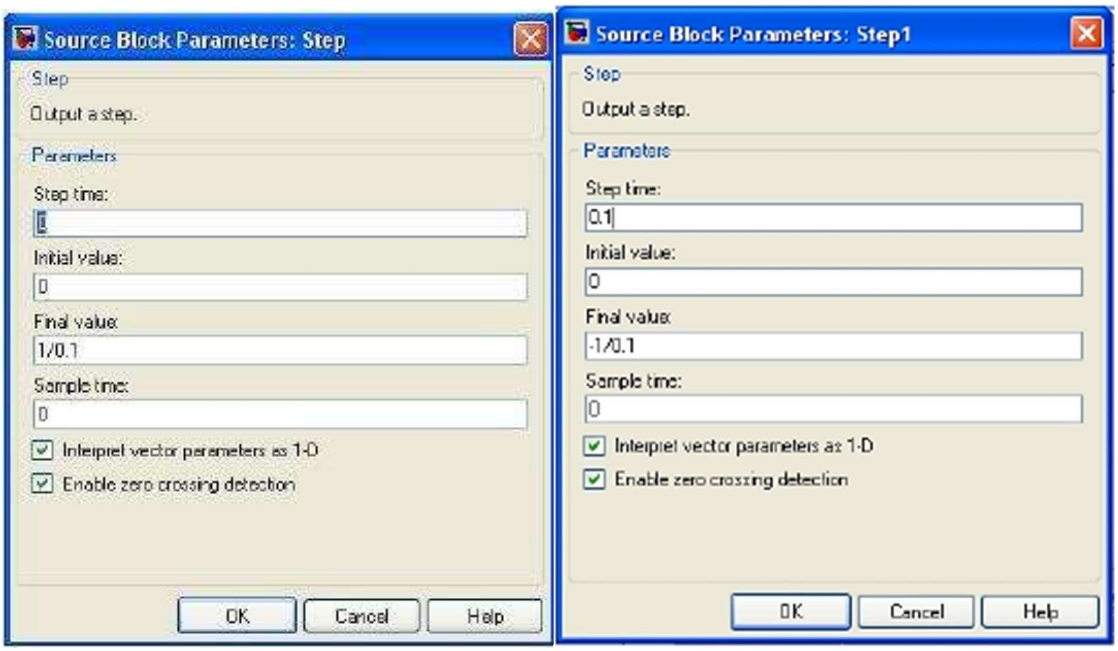
>> num = [1 5];
>> den = [1 3 2];
>> sys = tf (num, den)

Simulink n'a aucune source d'entrée impulsion mais il est possible de faire l'approximation d'une impulsion avec deux entrées échelons pour créer une seule impulsion de grande ampleur et très courte durée. Pour créer cette impulsion, le premier échelon commence à t=zéro et amplitude positive. Le deuxième échelon commence à t=T et amplitude négative. L'ampleur de chacun des échelons est égale à l'amplitude de l'impulsion souhaitée divisée par le temps de retard T.

Schéma fonctionnel dans simulink



Configuration du bloc des deux step

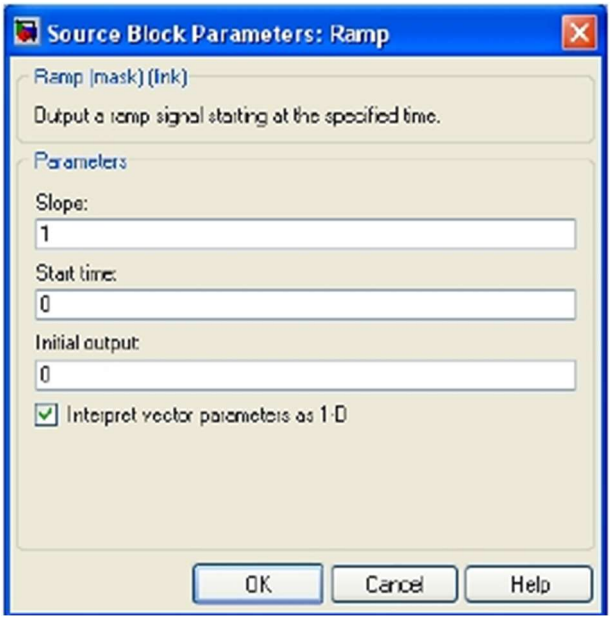


c) <u>Réponse à une rampe avec Simulink</u>

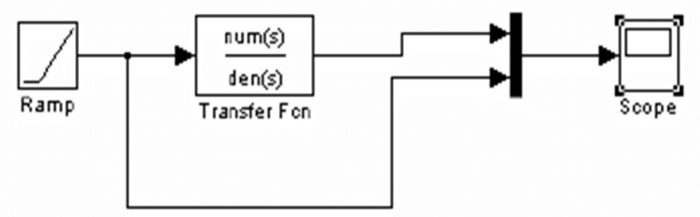
Nous devons déterminer la fonction de transfert dans l'espace de travail afin d'utiliser dans Simulink. Nous utiliserons la même fonction de transfert que l'exemple ci-dessus pour faire une comparaison des graphiques résultants.

>> den = [1 1 1];
>> num = [1];

Schéma fonctionnel dans Simulink



Configuration du bloc Ramp



TP N° 2 : Analyse des systèmes linéaires de 1er et 2ème ordre

Travail demandé ①: Analyse temporelle des systèmes linéaires du 1er et 2ème ordre

L'objectif de ce travail est d'étudier les systèmes de 1er et 2ème ordre dans le domaine temporel en utilisant MATLAB. Déterminer la réponse d'un système quelconque à des grandeurs d'entrée typique telles que l'impulsion, l'échelon et la rampe. Déterminer les caractéristiques d'une réponse telles que le temps de réponse $t_{r5\%,}$ le temps de montée t_m, le dépassement maximal D. Déterminer l'erreur en régime permanent associée soit à une grandeur d'entrée principale.

Travail sous MATLAB

I. On considère le système du premier ordre suivant *:*

$$F(P) = \frac{K}{1 + \tau p}$$

K : le gain statique.

$\boldsymbol{\tau}$: la constante du temps du système.

A. Réponse indicielle

1. Écrire un script qui permet de tracer les allures de ce système (***K=1*** et **τ =0.5**) pour les réponses indicielle s(t) par la commande **step.** Donner l'expression du temps de réponse à 5% t_r.
2. pour **τ=1** et ***K*** **=0.5**, **1.5 et 5**, écrire un script qui permet de tracer sur même figure la réponse indicielle par la commande **step** et mesurer le temps de réponse à 5%. Quel est l'effet de la valeur de ***K*** sur le temps de réponse $\mathbf{t_r}$ et l'erreur statique en régime permanent. Conclure.
3. Pour K **=1** et **τ=0.5, 1 et 1.5**, sur même figure relever la réponse indicielle et mesurer le temps de réponse à 5%. Quel est l'effet de la valeur de τ sur le temps de réponse tr et l'erreur statique en régime permanent. Conclure.

II. On considère le système du deuxième ordre suivant *:*

$$F(P) = \frac{k}{\frac{p^2}{\omega^2{}_n} + \frac{2\boldsymbol{m}}{\omega_n} p + 1}$$

$\boldsymbol{K}$: gain statique du système.

$\boldsymbol{\omega_n}$**:** La pulsation propre non amortie. Homogène à une pulsation et s'exprime en rad.s^{-1}.

$\boldsymbol{m}$: Le coefficient d'amortissement (sans dimension).

A. Réponse indicielle

1. Donner l'expression de la réponse indicielle s(t) face à un échelon unitaire. Discuter suivant les valeurs de $\boldsymbol{m}$.
2. Pour $\boldsymbol{m}$ **<1**, donner l'expression du : Temps de montée (t_m), le temps de réponse à 5% (t_r), le temps du pic (t_{pic}), le dépassement D en %, la pseudo-pulsation $\boldsymbol{\omega_p}$**.**

On prend ***k=1*** et $\boldsymbol{\omega_n}$**=10rd/s.**

3. Tracer sur une seule figure, la réponse indicielle de ce système pour $\boldsymbol{m}$ **=0.3, 0.5, 0.714, 1, 1.3.** Pour chaque réponse, relever le dépassement, le temps de pic, le temps de montée et l'erreur statique correspondante en donnant les résultats sous forme d'un tableau. Conclure.
4. Quel est les l'effet, pour la réponse indicielle d'une variation de la valeur **k=0.5** , **1.5 et 5** ($\boldsymbol{m}$ et $\boldsymbol{\omega_n}$ sont constantes) sur les paramètres de la question c. Conclure.
5. Quel est les l'effet, pour la réponse indicielle d'une variation de la valeur $\boldsymbol{\omega_n}$**=0.1** , **1 et 10** ($\boldsymbol{m}$ et k sont constantes) sur les paramètres de la question c. Conclure.

3) Réponse fréquentielle

L'analyse fréquentielle d'un système peut être effectuée facilement à l'aide des fonctions spécifiques, réalisant les tracés des lieux de Bode, Nyquist et Black-Nichols :

Diagramme de Bode.

Le diagramme de Bode peut être obtenu en écrivant:

>> bode(H)

>> bode(H,{w_deb,w_fin}) , il trace sur l'intervalle ,{w_deb,w_fin}

>> bode(H,w) , il trace sur l'intervalle définis par le vecteur w

>> bode(H1,H2,…….,w), idem que bode(H1,w), bode(H2,w)…….

>> [mod, phi, w] = bode(H,..), on récupère, sous forme de tableaux, le module, la phase et w (si elle n'a pas été fournie).

Diagramme de Nichols.

nichols : calcule H et et les trace dans un plan de Black.

L'utilisation est identique à celle de la fonction bode.

Diagramme de Nyquist.

nyquist : calcule Re(w) et Im(w) et les trace dans un plan de nyquist.

L'utilisation est identique à celle de la fonction bode.

Quelques fonctions d'analyse :

- margin(nom de la ft) : trace le lieu de Bode et indique dessus les marges de gain et phase. (autre syntaxe : [mG, mPhi, w_G, w_Phi]=margin(nom de ft)).
- pole : vec_p = pole (nom de la ft) calcule les pôles et les place dans vec_p.
- zero : vec_z = pole (nom de la ft) calcule les zéros et les place dans vec_z.
- pzmap(nom de la ft) :calcule les pôles et les zéros de la ft et les trace dans un plan complexe qui représente la carte des pôles et les zéros du système (autre syntaxe : [vec_p, vec_z] = pzmap(nom de ft)).
- grid, Noter que la commande grid produit une grille à l'écran.

Travail demandé ②: Analyse fréquentielle des systèmes linéaires du 1er et 2ème ordre

L'objectif de ce travail est d'étudier les systèmes de 1er et 2ème dans le domaine fréquentiel en utilisant MATLAB. Tracer les diagrammes de Bode, de Black et de Nyquist pour n'importe quel système.

I. On considère le système du premier ordre suivant *:*

$$G(P) = \frac{K}{1 + \tau p}$$

1. Donner l'expression du gain complexe $|G(j\omega)|$ ainsi que celle de $|G(\omega)|_{dB}$.
2. Donner l'expression de la phase $\varphi(j\omega)$. En déduire la pulsation de coupure du système ω_c.
3. On prend $\tau = 0.2$ et $K = 1$
4. Tracer alors les allures des courbes dans le diagramme de Bode, Nyquist et black. Retrouver les valeurs de τ et K.
5. Tracer avec Matlab le lieu des racines de G.

- Quel est l'effet d'une variation de la valeur du gain K ($\tau = cte$) sur le gain G, la phase φ et la pulsation de coupure.

- Quel est l'effet d'une variation de la valeur de τ ($K = cte$) sur sur le gain G, la phase φ et la pulsation de coupure.

II. On considère le système du deuxième ordre suivant *:*

$$G(P) = \frac{k}{\frac{p^2}{\omega^2{}_n} + \frac{2\boldsymbol{m}}{\omega_n} p + 1}$$

1. Donner l'expression du gain complexe $|G(j\omega)|$ ainsi que celle de $|G(\omega)|_{dB}$.
2. Donner l'expression de la phase $\varphi(j\omega)$, de la pulsation de résonance ω_R et du facteur de surtension M du système.

On prend m=0,2, $\omega_n = 10 rd/s$ *et* $K = 1$

3. Tracer alors les allures de Bode, Black et Nyquist de G.
4. En retraçant le lieu de Bode pour ω =[10^{-1} ; 10^{1}], chercher le maximum qui correspond à la résonance.
5. Tracer avec Matlab le lieu des racines de G.
6. visualiser les lieux de Bode, Black et Nyquist de G pour $\boldsymbol{m}$**=0.3; 0.5; 0.714; 1; 1.5**. Conclure.
7. Quel est l'effet d'une variation de la valeur du gain **K=0.5 ; 1.5 ;5** sur le gain G, la phase φ, la pulsation de résonance et la bande passante à 0dB.
8. Quel est l'effet d'une variation de la valeur de pulsation $\boldsymbol{\omega_n}$**=0.1 ; 1; 10** sur le gain G, la phase φ, la pulsation de résonance et la bande passante à 0dB.

Remarque : Au début de chaque script Matlab, il est préférable de commencer par nettoyer command window, effacer toutes les variables en Workspace et fermer toutes les fenêtres graphiques en tapant les commandes suivantes :

```
close all;
clear all;
clc
```

TP N° 3: Correcteurs P et PI

Objectif

L'objectif de la première partie de ce TP est de faire la synthèse d'un correcteur proportionnel P suivant un cahier des charges. La deuxième partie sert à étudier l'asservissement de la température d'un four par les correcteurs proportionnel P et proportionnel intégral PI et souligner l'intérêt de ces correcteurs.

Partie 01 : Synthèse d'un Correcteur Proportionnel :

Soit un système asservi de fonction de transfert en boucle ouverte donnée par:

$$G(p) = \frac{50}{p(1+p)\left(1+\frac{p}{40}\right)\left(1+\frac{p}{1000}\right)}$$

1) Analyse du système en BO et BF avant correction

a) Décrire sous Matlab la fonction de transfert G(p)

b) Donner les pôles de G(p), les représenter dans le plan complexe, ce système est-il stable?

c) Confirmez la réponse à la question précédente en observant la réponse indicielle de G(p).

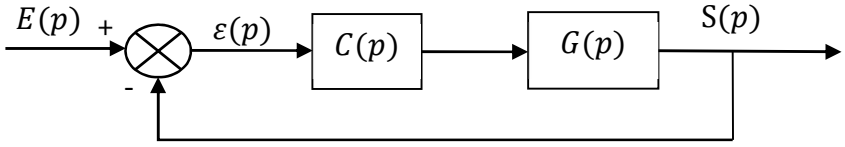
d) Tracer le diagramme de Bode de la FTBO G(p) sur un horizon de 10^{-3} à 10^{5}rd/s en précisant les pulsations de cassure et les pentes à l'origine et en haute fréquence et les déphasages en hautes et basses fréquences.

e) Est-ce que le bouclage unitairement du système peut améliorer son comportement et le rendre stable ? pour répondre à cette question, tracer Bode de G(p) ou voir les marges de gain et marge de phase ou tracer le lieu de Nyquist de G(p) (vérifier le critère de Nyquist). Conclure.

f) Réalisez le bouclage du système et décrire la fonction de transfert en boucle fermée F(p) sous Matlab puis retrouver sa réponse indicielle. Conclure.

2) Cahier des charges

On souhaite introduire ce système dans une boucle de correction afin d'améliorer ses performances. Le schéma de correction adopté est indiqué sur la figure suivante :



$T(p) = C(p)G(p)$: Fonction de transfert en boucle ouverte.

L'objectif est de synthétiser le correcteur *C(p)* qui permet l'obtention des caractéristiques suivantes :

- Marge de phase : $M\varphi \geq 45°$
- Erreur statique de position nulle : $\varepsilon(\infty) = 0$

Puisque la FT G(p) est de classe 1, l'erreur statique de position sera nulle: $\varepsilon(\infty) = 0$, il suffit donc d'introduire un correcteur proportionnel C(p)=Kp pour achever les performances décrites dans le cahier des charges.

3) Synthèse du correcteur

a) Déduire la fonction de transfert en boucle fermée et étudier la stabilité en fonction de kp en utilisant le critère algébrique de Routh.

Marge de phase : $M\varphi = ArgT(j\omega_1) - (-180°) = ArgT(j\omega_1) + 180°$ avec $|T(j\omega_1)| = 1$

Pour avoir $M\varphi \geq 45°$, il faut que $ArgT(j\omega_1) \geq -135°$

b) Déterminer ω_2 qui permet d'avoir $[G(j\omega_2)] = -135°$.
c) Mesurer le module de$|G(j\omega_2)|$ et calculer le gain Kp.
d) Saisir la FTBO corrigée T(p). Tracer son lieu de Bode et chercher la marge de gain et de phase du nouveau système. Avez-vous la marge souhaitée ? si non, vérifiez vos calculs.
e) Tracer le lieu de Nyquist de la FTBO corrigée $T(p) = KpG(p)$, pour voir si le critère de Nyquist est bien vérifié.

4) Analyse du système en BF après correction

a) Réalisez le bouclage du système corrigé et décrire la fonction de transfert en boucle fermée F(p) sous Matlab puis retrouver sa réponse indicielle.
b) Déterminer : le gain statique, le temps de réponse t_r à 5%, le dépassement **D%,** le temps de pic t_{pic} et l'erreur statique de position $\varepsilon_p(\infty)$.

Partie 02 : Asservissement de température par des correcteurs proportionnel P et proportionnel intégral PI

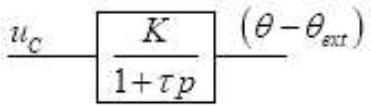
Détermination de la transmittance du système et préparation :

Un système de chauffage qui doit faire passer la température d'une enceinte de θe (température extérieure) à θ_f. La puissance fournie est proportionnelle à la tension de commande U_c et l'énergie fournie se répartit comme suit :

- énergie permettant de chauffer l'enceinte telle que $dW_2 = M.C.d(\theta_f - \theta_e)$
- énergie perdue par pertes de l'enceinte telle que $P_3 = \frac{dW_3}{dt} = \frac{\theta_f - \theta_e}{R_{th}}$

Pour une tension de commande de 5V, la puissance totale fournie par la résistance de chauffage est de P1=1kW.
La température se stabilise à 80°C (On atteint 95 % de la valeur max au bout de 3 minutes) pour une température extérieure de 20°C.

a) Écrire l'équation différentielle vérifiée par l'écart de température.
b) Déterminer la transmittance en boucle ouverte et la mettre sous la forme avec les CI=0:



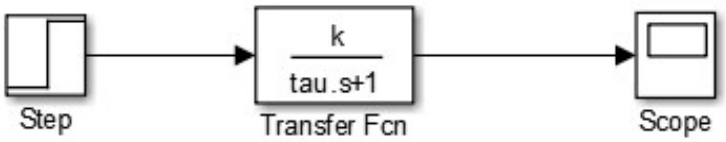
c) À partir des données de l'énoncé, calculer K et τ.
d) Quelle serait la température pour une tension de commande U_0=3V?
e) Au bout de combien de temps serait-elle obtenue à 5% près ?

1) Simulation sous Matlab

Retrouver la réponse indicielle du système sous Matlab.
Avant de quitter Matlab, sauvez le travail effectué sous un fichier de type Initiation.m

2) Simulation sous Simulink:

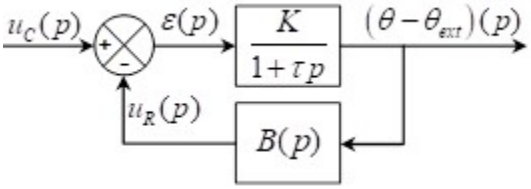
a) Placer les blocs suivants :



La fonction de transfert est celle déterminée précédemment, la sortie est enregistrée comme une variable de sortie.

b) Simuler la montée en température suite à une tension de commande U_c en échelon et observer la réponse temporelle sur 400s et déterminer le temps de réponse $t_{r5\%}$.

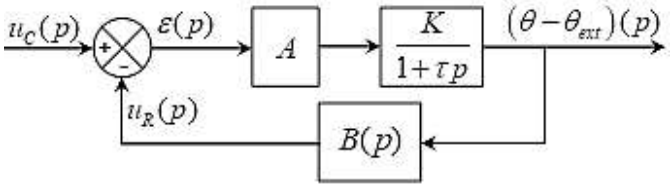
3) Bouclage du système.



<u>Calculs préliminaires et Préparation</u>

Dans le but de stabiliser la température, on boucle le système par un ensemble thermocouple-amplificateur tel que B=0.05 V / °C.

Dans le but d'améliorer ce système, on ajoute un correcteur proportionnel A.



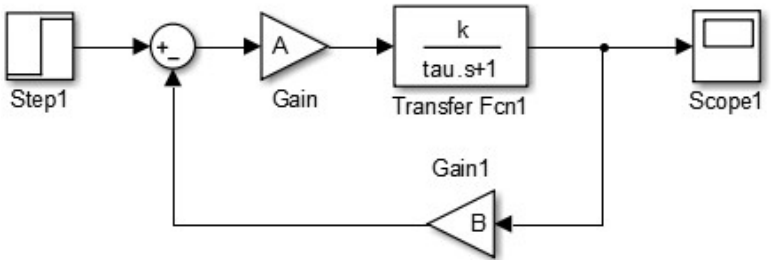
a) Quelle est la nouvelle transmittance pour A=1 et A=10? (constante de temps τ' et gain statique K')

b) Quelle tension de commande U_c sera nécessaire pour avoir une température de 80°C et quelle sera la nouvelle constante de temps du système ?

c) Calculer l'erreur statique et le temps de réponse $t_{r5\%}$ dans ce cas.

4) Simulation du système bouclé : influence du correcteur proportionnel

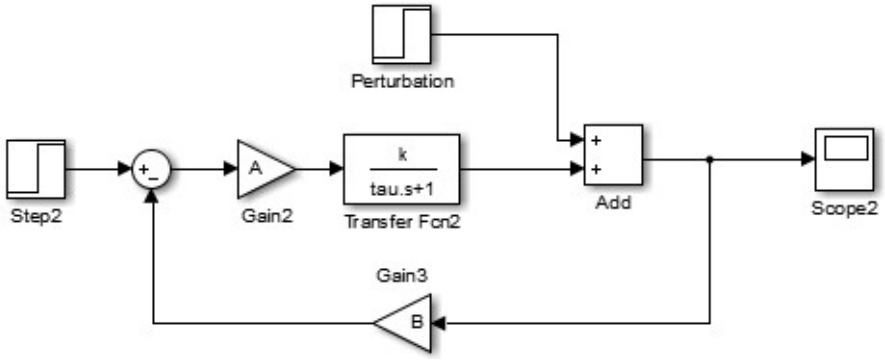
a) Modifier le schéma bloc avec ce nouveau correcteur et effectuer le bouclage dans Simulink



b) Observer la réponse de votre système à la commande U_c pré-calculée.

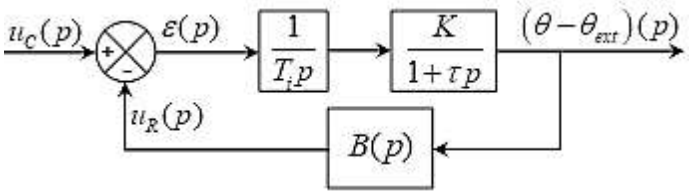
c) Confirmer vos calculs : constante de temps et gain.

d) Placer une perturbation et observer l'influence du correcteur pour une perturbation en échelon (amplitude +1) commençant à 100s, la simulation doit se faire sur 600s.

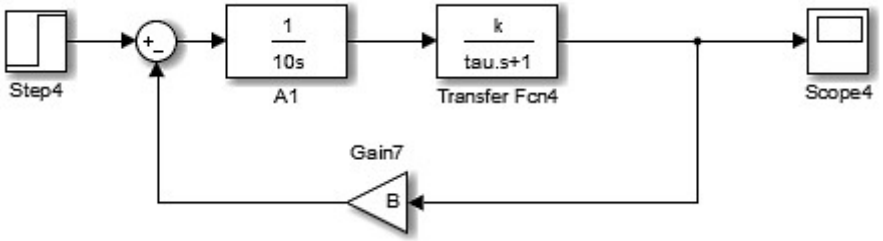


5) Simulation du système bouclé : influence du correcteur intégral

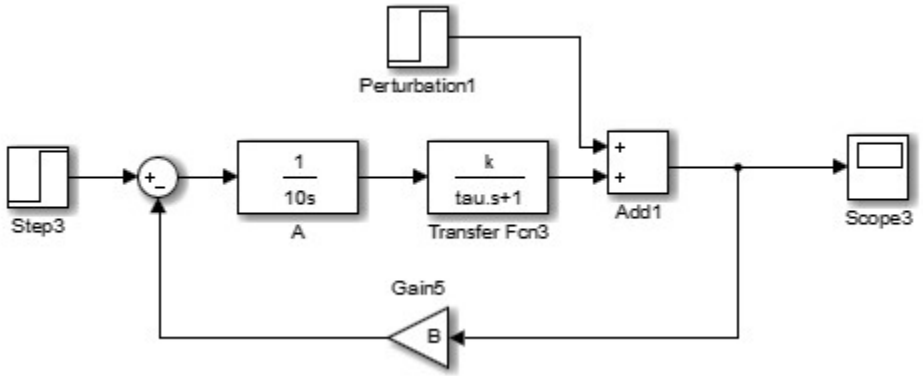
Au lieu d'un correcteur uniquement proportionnel, on met un correcteur proportionnel intégral PI tel que $A = \frac{1}{10p}$, déterminer la nouvelle transmittance, la tension de commande U_c pour avoir 80 °C et la valeur de l'erreur statique et le temps de réponse $t_{r5\%}$. Conclure.



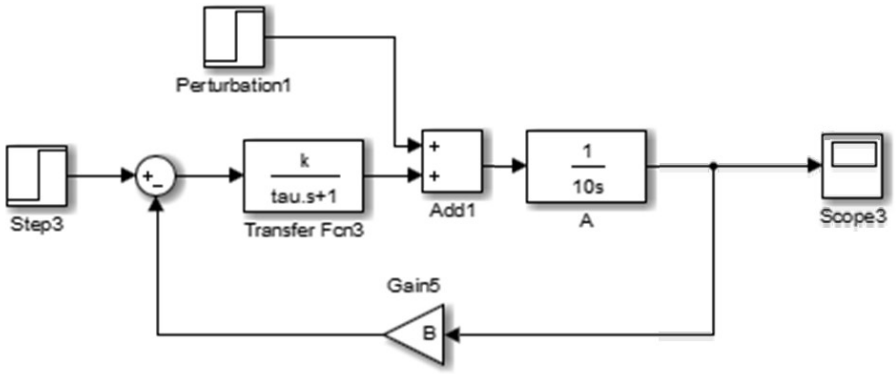
a) Confirmer les calculs précédents sous Simulink



b) Observer l'effet de la perturbation, pour une perturbation en échelon (amplitude +10) commençant à 400s, la simulation doit se faire sur 1000s.



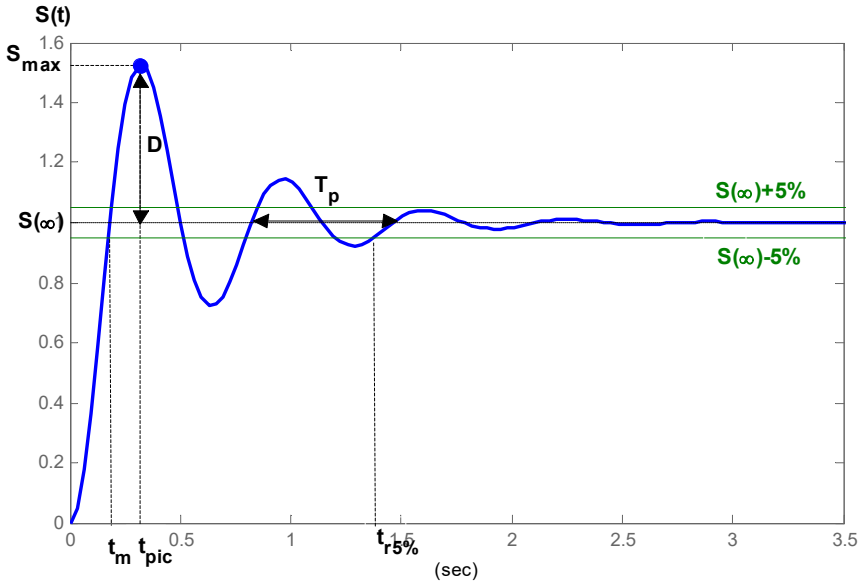
c) Observer l'effet de la perturbation en modifiant le schéma pour obtenir un équivalent correspondant au schéma-bloc suivant :



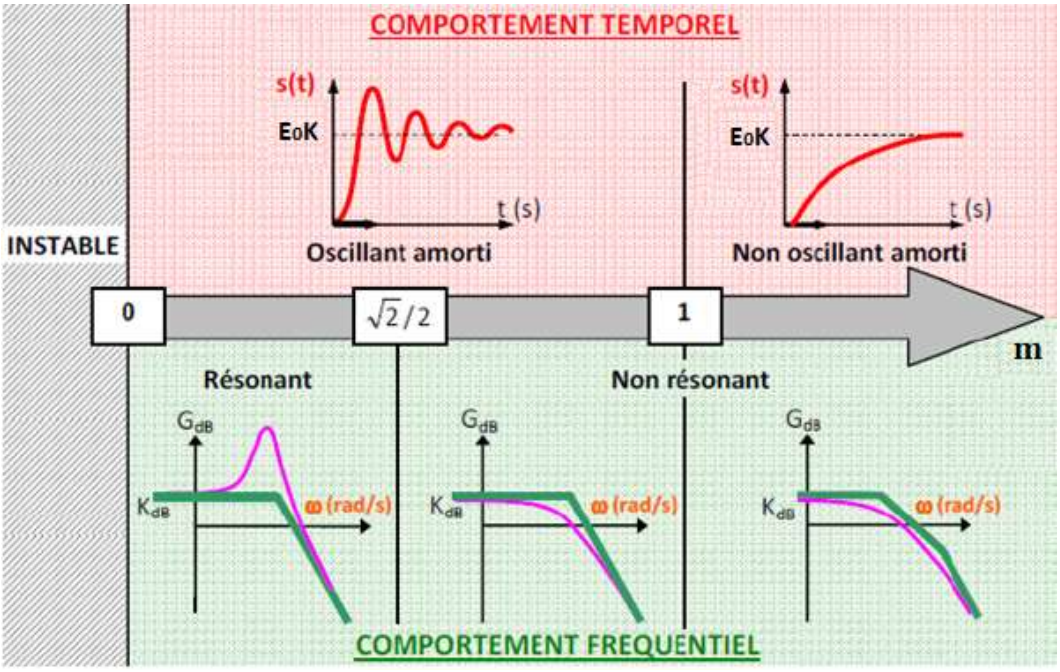
Le résultat vous surprend-il ? Expliquez la différence entre cette simulation et celle réalisée précédemment.

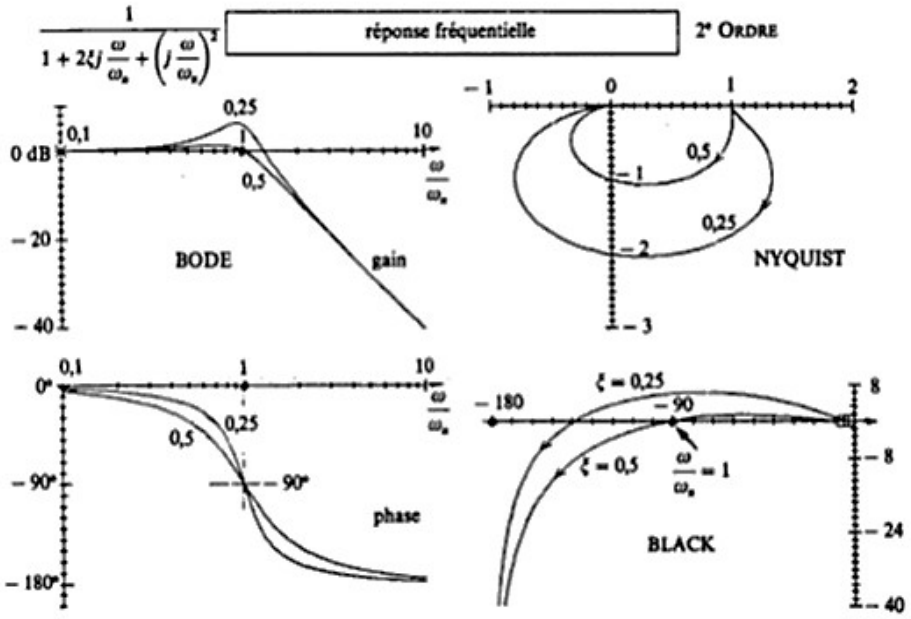
ANNEXE 1. Systèmes linéaires du second ordre.

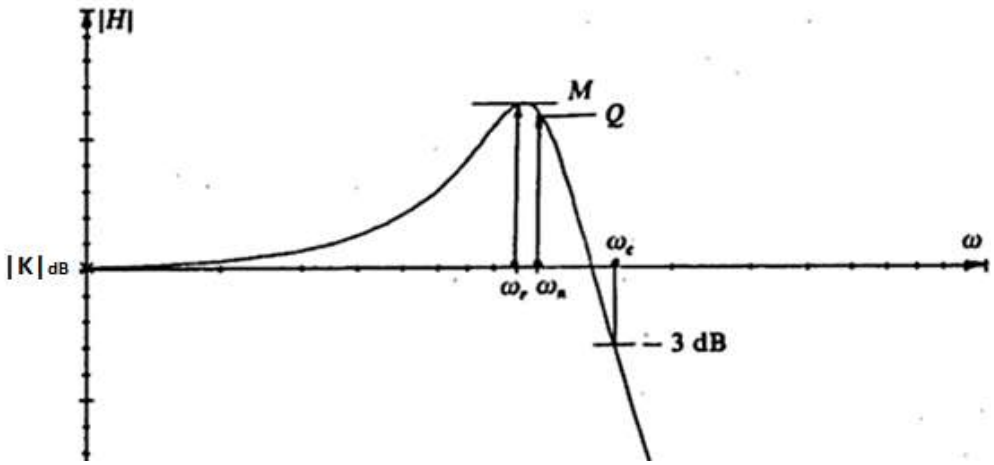
$$H(P) = \frac{S(p)}{E(p)} = \frac{k}{\frac{p^2}{\omega^2{}_n} + \frac{2m}{\omega_n}p + 1}$$



Pulsation de résonance	$\omega_r = \omega_0\sqrt{1-2m^2}$
Pulsation de coupure	$\omega_c = \omega_0\sqrt{1-2m^2+\sqrt{1+(1-2m^2)}}$
Facteur de résonance	$M = \frac{1}{2m\sqrt{1-m^2}}$
Facteur de qualité	$Q = \frac{1}{2m}$
Temps de montée	$t_m = \frac{1}{\omega_0\sqrt{1-m^2}}\left(\pi - \frac{1}{\cos m}\right)$
Temps de réponse à n %	$t_r = \frac{1}{\omega_0 m} log\left(\frac{100}{n}\right)$
Temps de pic	$t_{pic} = \frac{\pi}{\omega_0\sqrt{1-m^2}} = \frac{T_p}{2}$
Pseudo période	$T_p = \frac{2\pi}{\omega_0\sqrt{1-m^2}}$
Dépassement	$D(\%) = 100e^{\frac{-\pi m}{\sqrt{1-m^2}}}$
Nombre d'oscillations complète	$n = \frac{1}{2m}$


COMPORTEMENT TEMPOREL
s(t)
EoK
t (s)
Oscillant amorti
s(t)
EoK
t (s)
Non oscillant amorti
INSTABLE
0
√2/2
1
m
Résonant
Non résonant
G_dB
K_dB
ω (rad/s)
COMPORTEMENT FREQUENTIEL


réponse fréquentielle
2e ORDRE
BODE
gain
NYQUIST
phase
BLACK
0 dB
−20
−40
0,1
1
10
0,25
0,5
−90°
−180°
ξ = 0,25
ξ = 0,5
−180
−90
−8
−24
−40


|H|
M
Q
|K| dB
ω_c
ω
ω_r
ω_n
− 3 dB